

Corso di Laurea Magistrale in Ingegneria e Scienze Informatiche

Modernizing a Spark-Based ETL Platform Using Transactional Lakehouse Technologies on AWS

Relatore

Prof. Boschetti Marco Antonio

Candidato

Tommaso Brini

Correlatori

Dott. CoSupervisor 1

Dott. CoSupervisor 2

Azienda

PROMETEIA S.P.A

Abstract

Questa tesi analizza la modernizzazione di una piattaforma ETL basata su Apache Spark e AWS Glue, precedentemente vincolata a un'architettura Data Lake tradizionale sul formato Parquet. L'obiettivo centrale del lavoro è l'introduzione del paradigma Transactional Lakehouse tramite l'adozione di Apache Iceberg. L'introduzione di questo table format ha permesso di abilitare proprietà ACID, funzionalità di time travel e una gestione avanzata dei metadati.

Nel corso del tirocinio è stato progettato e implementato un prototipo in grado di gestire pipeline complesse (INSERT, UPDATE, DELETE e MERGE) su dataset di grandi dimensioni, includendo una strategia automatizzata per la sincronizzazione del catalogo e dei metadati.

La soluzione proposta viene quindi confrontata con l'architettura esistente per valutarne gli effetti in termini di prestazioni, scalabilità e robustezza operativa, evidenziando vantaggi e limiti delle tecnologie lakehouse nel contesto di pipeline ETL in ambiente cloud.

Contents

Abstract	iii
1 Introduzione	1
2 Stato dell'arte e tecnologie	5
2.1 Evoluzione delle Architetture Dati	5
2.1.1 Il Data Warehouse Tradizionale	5
2.1.2 L'avvento del Data Lake	6
2.1.3 Paradigma Data Lakehouse	8
2.2 Open File Format e Apache Parquet	10
2.2.1 L'affermazione di Apache Parquet come standard	10
2.2.2 I limiti di un approccio basato solo su File Format	11
2.3 Apache Spark e l'elaborazione distribuita	13
2.4 Open Table Format e paradigma Lakehouse	14
2.4.1 Differenza tra File Format e Table Format	14
2.4.2 Il ruolo del Metadata Layer	15
2.4.3 Funzionalità Chiave dell'Architettura	15
2.5 Apache Iceberg: Architettura e funzionalità	17
2.5.1 Architettura dei metadata	17
2.5.2 Operazioni transazionali e Snapshot	17
2.5.3 Time Travel e Schema Evolution	18
2.5.4 Maintenance e ottimizzazione	18
2.5.5 Confronto con Delta Lake e Hudi	18
2.6 Ecosistema AWS	18
3 Analisi As-is: Architettura Esistente	19
4 Progettazione e Implementazione	21
5 Test e risultati	23
6 Conclusioni e sviluppi futuri	25

CONTENTS

Bibliography	27
--------------	----

Chapter 1

Introduzione

Il presente lavoro di tesi è stato sviluppato nell’ambito di un tirocinio svolto presso Prometeia S.p.A., azienda attiva nella consulenza e nello sviluppo di soluzioni software per il settore finanziario.

Negli ultimi anni la crescita esponenziale dei dati generati da applicazioni distribuite, sistemi transazionali e servizi cloud ha reso necessaria l’evoluzione delle architetture aziendali per la gestione e l’analisi dei dati. In questo contesto, i processi di **Extract, Transform, Load (ETL)** hanno assunto un ruolo centrale nelle pipeline di data engineering, richiedendo soluzioni sempre più scalabili, affidabili e flessibili.

La diffusione dei paradigmi Big Data ha favorito l’adozione dei Data Lake, architetture basate su storage distribuito in grado di memorizzare e gestire grandi quantità di dati eterogenei a costi ridotti. Formati colonnari come Apache Parquet hanno quindi rappresentato uno standard de facto per l’elaborazione distribuita tramite framework come Apache Spark. Nel modello Lakehouse, i Data Lake moderni si basano infatti su storage cloud economico e formati aperti per supportare workload analitici e di machine learning. [AGX⁺21] [BGK21]

Nonostante i vantaggi in termini di costo e scalabilità, l’approccio tradizionale basato su file Parquet presenta alcune limitazioni significative. In particolare, i Data Lake classici non forniscono nativamente proprietà transazionali ACID, una gestione evoluta dei metadati o un supporto efficiente per operazioni di aggiornamento incrementale. Operazioni come UPDATE, DELETE e MERGE richiedono

quindi meccanismi applicativi complessi, spesso basati sulla riscrittura di intere partizioni o dataset, con un aumento della complessità delle pipeline ETL e del rischio di inconsistenze nei dati.

Per superare tali criticità si è diffuso il paradigma del Data Lakehouse, che combina la flessibilità dei Data Lake con caratteristiche tipiche dei Data Warehouse, introducendo transazioni, versionamento dei dati, schema evolution e una gestione avanzata dei metadati. [Sah24]

Tra le tecnologie emergenti in questo ambito, Apache Iceberg si è affermato come uno dei principali table format open-source per la gestione di dataset analitici su larga scala. Iceberg introduce un layer di metadata management che separa la struttura logica della tabella dalla disposizione fisica dei file su storage object-based, consentendo la gestione di snapshot consistenti, time travel e operazioni incrementali transazionali. [Apa25a]

A differenza di un approccio puramente file-based, Iceberg mantiene lo stato della tabella attraverso metadata versionati e manifest file, consentendo aggiornamenti atomici mediante commit espliciti. [Apa25b]

Questa tesi si inserisce nel contesto della modernizzazione di una piattaforma ETL aziendale basata su Apache Spark e servizi AWS. L'architettura esistente utilizza file Parquet memorizzati su Amazon S3 ed elaborati tramite AWS Glue, presentando criticità legate alla gestione degli aggiornamenti incrementali, alla sincronizzazione del catalogo e alla garanzia di idempotenza dei processi batch.

L'obiettivo del lavoro è progettare e valutare una nuova architettura basata sul paradigma Transactional Lakehouse attraverso l'introduzione di Apache Iceberg. In particolare, il lavoro analizza l'integrazione del table format con Apache Spark e l'ecosistema AWS, includendo aspetti relativi alla gestione dei metadati, all'esecuzione di operazioni MERGE transazionali e ai processi di maintenance delle tabelle. Durante il progetto sono inoltre state esplorate le funzionalità offerte da **Amazon S3 Tables** [Ama25a], soluzione managed introdotta da AWS per la gestione di tabelle Iceberg.

Il contributo principale della tesi consiste nella progettazione e implementazione di un prototipo in grado di eseguire operazioni incrementali affidabili e idempotenti su dataset di grandi dimensioni, confrontando la nuova soluzione con l'architettura preesistente in termini di prestazioni, scalabilità e robustezza.

L'analisi sperimentale evidenzia inoltre i trade-off introdotti dall'adozione del paradigma lakehouse, valutando vantaggi e limiti sia di un'implementazione Iceberg tradizionale sia dell'approccio gestito offerto da S3 Tables.

Nei prossimi capitoli, la tesi introdurrà lo stato dell'arte relativo alle architetture Data Lake e Lakehouse, descrivendo le principali tecnologie utilizzate (Capitolo 2). Successivamente, verrà presentata un'analisi dettagliata dell'architettura esistente e delle criticità riscontrate (Capitolo 3), seguita dalla progettazione e implementazione della nuova soluzione basata su Apache Iceberg (Capitolo 4). Infine, i risultati sperimentali e i benchmark effettuati saranno discussi nel Capitolo 5, raccogliendo conclusioni e possibili sviluppi futuri nel Capitolo 6.

Chapter 2

Stato dell'arte e tecnologie

2.1 Evoluzione delle Architetture Dati

L'evoluzione delle architetture dati è strettamente legata alla crescita del volume, della varietà e della velocità con cui le informazioni vengono generate e processate all'interno dei sistemi informativi moderni.

La necessità di estrarre valore da sorgenti dati eterogenee ha guidato l'evoluzione architetturale dai tradizionali database relazionali fino ai moderni ecosistemi distribuiti.

Nel corso degli anni, le piattaforme analitiche hanno attraversato diverse fasi evolutive, passando dai sistemi relazionali tradizionali ai Data Warehouse, successivamente ai Data Lake e, più recentemente, al paradigma Lakehouse.

2.1.1 Il Data Warehouse Tradizionale

Nel contesto dei tradizionali sistemi di gestione di database relazionali (RDBMS), le piattaforme sono state storicamente suddivise in sistemi per l'*Online Transactional Processing* (OLTP) e sistemi per l'*Online Analytical Processing* (OLAP).

I primi sistemi OLAP per la gestione dati per finalità analitiche erano basati su architetture di tipo **Data Warehouse**. Tali sistemi si fondano su schemi relazionali fortemente strutturati e su processi ETL volti a trasformare e normalizzare i dati prima del caricamento.

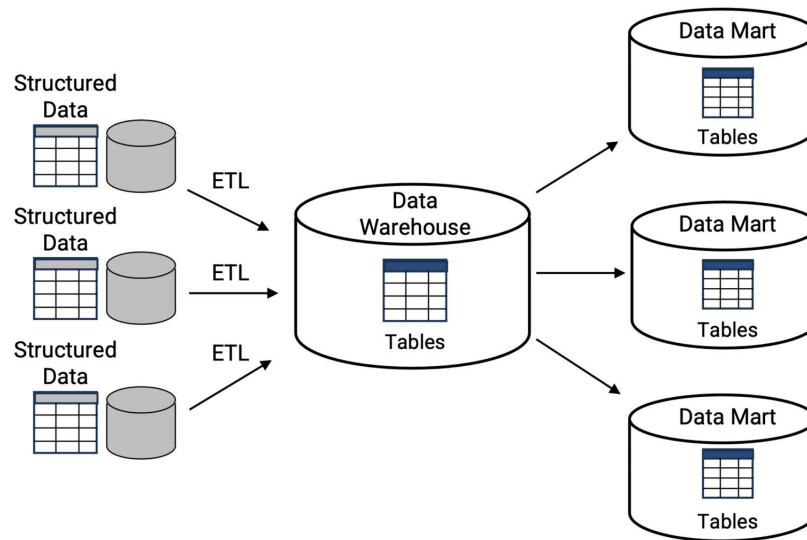


Figure 2.1: Data Warehouse Architecture. [Nix24]

Questo approccio garantisce elevata affidabilità, proprietà ACID e buone performance per reporting e business intelligence. Inoltre, questi sistemi garantiscono ottime prestazioni di interrogazione grazie all'utilizzo di partizionamenti e indicizzazioni avanzate.

Tuttavia, l'imposizione di formati di dati strettamente strutturati ne limita fortemente la flessibilità rispetto alla gestione di dati eterogenei e workload avanzati di analytics e machine learning. I costi di archiviazione su queste piattaforme sono molto elevati, l'infrastruttura risulta rigida di fronte alla necessità di scalare computazionalmente e l'architettura chiusa porta frequentemente al fenomeno del *vendor lock-in*.

2.1.2 L'avvento del Data Lake

Per superare l'incapacità dei Data Warehouse di gestire dati non strutturati e semi-strutturati e con l'ulteriore aumento delle quantità di dati prodotti da applicazioni web, dispositivi mobili e sistemi distribuiti, parallelamente alla diffusione delle tecnologie Big Data è emerso il concetto di **Data Lake**.

Questa architettura sfrutta file system distribuiti come *Hadoop HDFS* e, più recentemente, sistemi di archiviazione a oggetti in cloud a basso costo (come Amazon S3), permettendo di memorizzare enormi quantità di dati grezzi a costi estremamente contenuti. In questo contesto, l'introduzione di formati di file aperti e colonnari, come Apache Parquet e ORC, ha portato notevoli benefici in termini di interoperabilità e prestazioni di interrogazione all'interno del Data Lake.

La differenza principale rispetto al Data Warehouse è quindi la possibilità di memorizzare dati strutturati, semi-strutturati e non strutturati senza la necessità di un processo di trasformazione preventiva, garantendo una maggiore flessibilità e scalabilità.

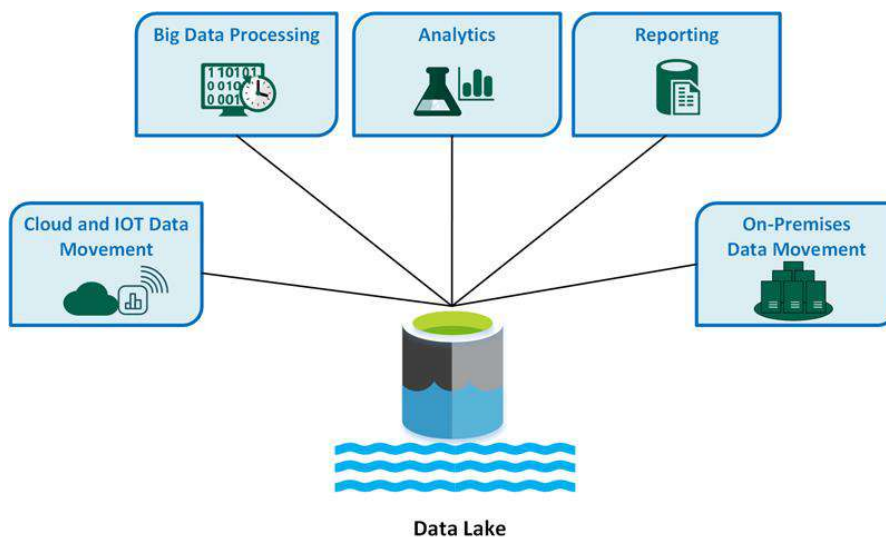


Figure 2.2: Data Lake Architecture. [el]

Nonostante l'enorme flessibilità e i vantaggi in termini di costo e capacità di ingestione, i Data Lake tradizionali hanno mostrato importanti limitazioni operative, che spesso li trasformano in ecosistemi difficilmente gestibili, noti come **Data Swamp** (letteralmente, paludi di dati).

In particolare, l'incapacità di imporre uno schema rigido e l'impossibilità di garantire transazioni ACID espongono il sistema a frequenti rischi di corruzione dei dati in caso di fallimenti dei job di scrittura. Inoltre, l'assenza di funzionalità

transazionali, la gestione limitata dei metadati e la difficoltà nel supportare operazioni incrementali hanno spesso portato alla creazione di pipeline ETL complesse e difficili da mantenere. Inoltre, molte organizzazioni hanno adottato architetture ibride in cui i dati venivano continuamente replicati tra Data Lake e Data Warehouse, aumentando costi, latenza e complessità gestionale.

2.1.3 Paradigma Data Lakehouse

Il Data Lakehouse è il concetto architettonico nato per superare i limiti intrinseci di Data Warehouse e Data Lake, offrendo una soluzione dati unificata. Questa nuova architettura riesce a fondere i due mondi, portando la conformità ACID e l'affidabilità dei dati tipiche dei Data Warehouse sopra l'archiviazione a basso costo, scalabile e interoperabile dei Data Lake.

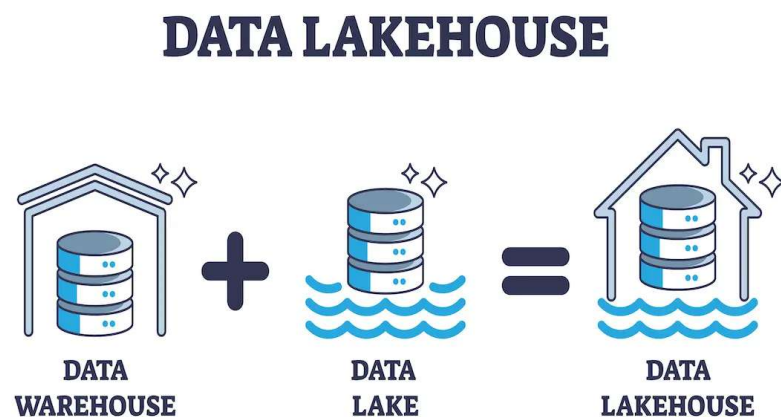


Figure 2.3: Data Lakehouse Architecture. [Ach24]

Un Lakehouse è una piattaforma basata su formati aperti e storage object-based che fornisce supporto nativo per analytics, machine learning e gestione transazionale dei dati. [AGX⁺21]

Le architetture Lakehouse introducono infatti caratteristiche quali:

- proprietà ACID;
- gestione evoluta dei metadati;
- versionamento e time travel;
- supporto a operazioni UPDATE, DELETE e MERGE;
- interoperabilità tra differenti motori di elaborazione.

Tali funzionalità vengono generalmente implementate tramite Open Table Format, come **Apache Iceberg**, Delta Lake e Apache Hudi, che aggiungono un layer logico di gestione sopra i file presenti nel Data Lake.

Questo approccio consente di mantenere la flessibilità e il basso costo dello storage object-based, introducendo al contempo meccanismi di consistenza e affidabilità tipici dei sistemi warehouse tradizionali.

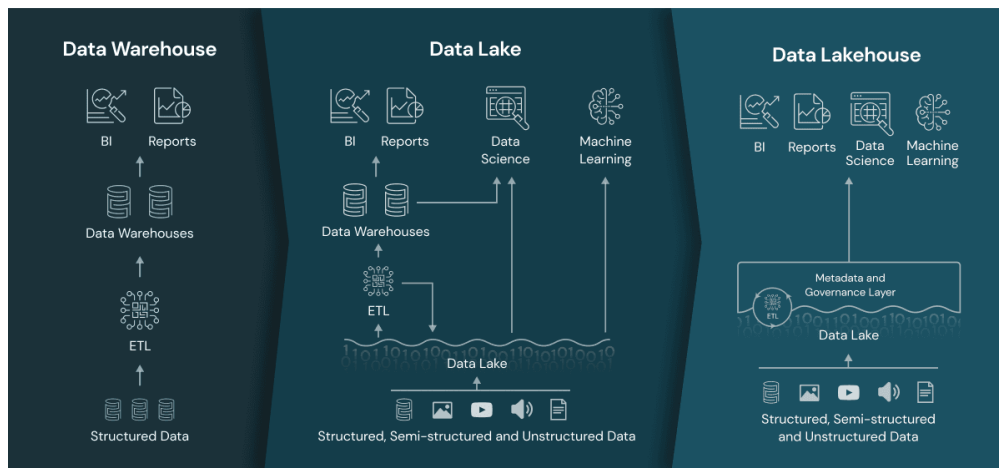


Figure 2.4: Data Architectures. [Sta24]

2.2 Open File Format e Apache Parquet

La diffusione delle architetture Big Data e dei Data Lake ha favorito l'adozione di *Open File Format* progettati per supportare l'elaborazione distribuita di grandi volumi di dati. L'obiettivo principale di questi open format è garantire l'interoperabilità tra diversi motori di calcolo e piattaforme cloud, evitare costose duplicazioni dei dati, ridurre i costi di archiviazione e ottimizzare le prestazioni di accesso alle informazioni.

All'interno di questo panorama, i formati si dividono principalmente in due categorie: orientati per riga (Row-oriented) e orientati per colonna (Column-oriented). Mentre i formati per riga (come Apache Avro o i tradizionali CSV) sono ideali per i sistemi transazionali tipici dei DBMS, i formati colonnari sono stati introdotti nei sistemi Big Data per massimizzare le performance analitiche.

2.2.1 L'affermazione di Apache Parquet come standard

Tra i formati maggiormente diffusi in ambito analitico vi è Apache Parquet, un formato colonnare open-source sviluppato per ottimizzare le performance di lettura e compressione nei workload analitici distribuiti. Parquet è stato progettato per integrarsi con ecosistemi Big Data come Apache Hadoop e Apache Spark, diventando progressivamente uno standard de facto per la memorizzazione di dataset nei Data Lake. [Apa25c]

A differenza dei tradizionali formati row-based, nei quali i dati vengono memorizzati riga per riga, Parquet utilizza un'organizzazione column-oriented. In un formato row-based, i valori appartenenti alla stessa riga vengono salvati consecutivamente sul disco, rendendo efficiente l'accesso transazionale ai record completi. Tuttavia, nei workload analitici *OLAP* è spesso necessario leggere solo un sottoinsieme delle colonne disponibili; in tali scenari, l'approccio columnar consente di ridurre significativamente il volume di dati letto da storage e migliorare le performance delle query.

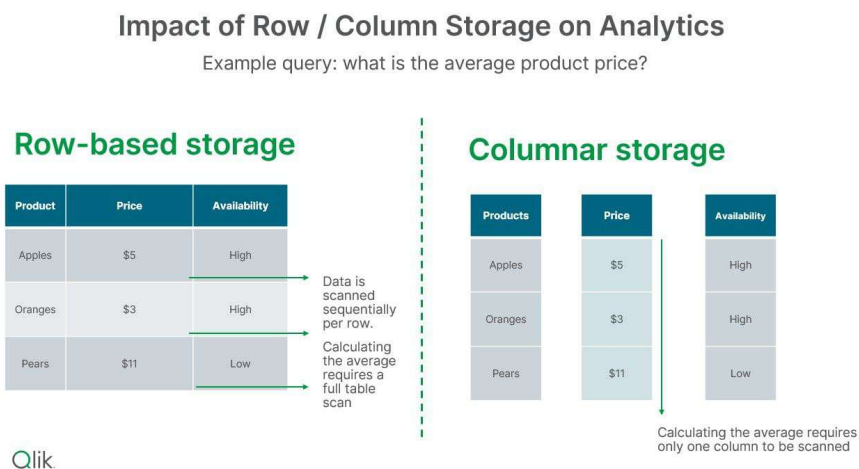


Figure 2.5: Row-oriented vs Column-oriented storage. [Sta25]

L'organizzazione colonnare offre inoltre vantaggi rilevanti in termini di compressione dei dati. Poiché i valori appartenenti alla stessa colonna tendono ad avere struttura e distribuzione simili, algoritmi di encoding e compressione risultano più efficaci rispetto ai formati row-based. Questo permette di ridurre i costi di storage e migliorare l'efficienza delle operazioni di scansione.

Inoltre, un ulteriore elemento che ha favorito l'adozione di Parquet è la sua natura open-source e il supporto per un ampio ecosistema di strumenti e piattaforme cloud, garantendo interoperabilità e flessibilità nella gestione dei dati. Infatti, il formato è supportato nativamente da numerosi framework e query engine, tra cui **Apache Spark**, **Amazon Athena**, **Trino**. Tale caratteristica ha reso possibile la costruzione di Data Lake aperti e indipendenti dal motore di elaborazione utilizzato.

2.2.2 I limiti di un approccio basato solo su File Format

Nonostante questi vantaggi, l'utilizzo di semplici file Parquet presenta alcune limitazioni significative nei moderni workload Lakehouse. Parquet è infatti un file

format e non un table format: esso definisce esclusivamente il modo in cui i dati vengono memorizzati all'interno dei file, senza fornire funzionalità avanzate di gestione transazionale o metadata management.

Nello specifico, un Data Lake basato esclusivamente su Parquet non supporta nativamente:

- **proprietà ACID**
- **operazioni UPDATE e DELETE efficienti**
- **gestione transazionale concorrente**
- **versionamento dei dati e time travel**
- **schema evolution avanzata**

Queste limitazioni hanno portato alla nascita degli **Open Table Format**, tecnologie progettate per estendere le capacità dei file format tradizionali introducendo gestione avanzata dei metadati, snapshot transazionali e supporto a operazioni incrementali. Tra le principali soluzioni emerse in questo ambito vi sono Apache Iceberg, Delta Lake e Apache Hudi, che rappresentano il fondamento delle moderne architetture Lakehouse.

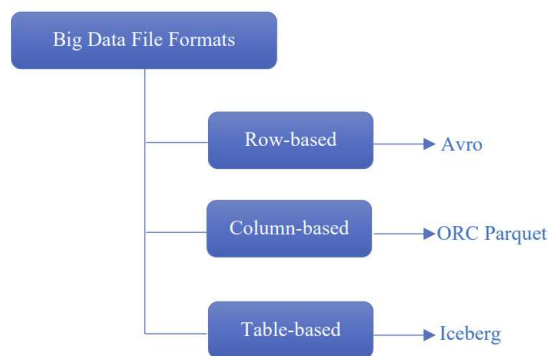


Figure 2.6: Different File Formats [NS25]

2.3 Apache Spark e l'elaborazione distribuita

La crescente diffusione dei Big Data richiede motori computazionali in grado di scalare orizzontalmente in modo efficiente. In questo contesto, Apache Spark rappresenta uno dei principali framework utilizzati nelle moderne architetture Data Lake e Lakehouse grazie alle sue capacità di parallelizzazione, scalabilità e integrazione con differenti sistemi di storage. [Apa25d]

Apache Spark è un framework open-source per il calcolo distribuito, progettato per garantire elevate prestazioni attraverso l'elaborazione in-memory. Rispetto ai precedenti paradigmi, come il classico modello MapReduce, Spark riduce drasticamente le operazioni di lettura e scrittura su disco, risultando particolarmente adatto a workload analitici e pipeline ETL complesse.

L'architettura di Spark si basa su un modello master-worker. Il processo centrale, chiamato Driver, ospita la logica applicativa e coordina l'esecuzione distribuendo i task ai vari Executor eseguiti sui nodi Worker del cluster. Questi ultimi elaborano i dati in parallelo sfruttandone la suddivisione in partizioni distribuite, massimizzando l'efficienza computazionale.

L'interazione con il framework avviene tipicamente attraverso API ad alto livello, in particolare DataFrame e Spark SQL. Tali interfacce consentono di manipolare dataset distribuiti in forma tabellare mediante un approccio dichiarativo, simile a quello dei database relazionali tradizionali.

Grazie alla sua natura distribuita, Spark può integrarsi con differenti sistemi di storage e formati dati, tra cui Apache Parquet, Apache Iceberg e object storage cloud come Amazon S3. Questa interoperabilità ha favorito la sua diffusione come motore principale per processi ETL e workload analitici in ambienti cloud-native.

Spark si occupa esclusivamente di elaborare i dati e non fornisce nativamente proprietà transazionali ACID. Per garantire atomicità, consistenza e affidabilità nelle operazioni distribuite, è quindi necessario integrare il motore computazionale con un layer logico avanzato di gestione delle tabelle, rappresentato dagli Open Table Format.

2.4 Open Table Format e paradigma Lakehouse

Il paradigma Lakehouse trova la sua effettiva realizzabilità tecnica nell'introduzione di un livello software intermedio noto come *Open Table Format*. Questo strato logico si pone l'obiettivo di colmare il divario tra la flessibilità dell'archiviazione di file su object storage e la rigorosa gestione tabellare tipica dei sistemi relazionali tradizionali.

2.4.1 Differenza tra File Format e Table Format

L'evoluzione delle moderne architetture Data Lake ha evidenziato i limiti dei tradizionali file format nella gestione di workload analitici sempre più complessi. Sebbene formati colonnari come Apache Parquet offrano elevate prestazioni, essi definiscono esclusivamente la rappresentazione fisica dei dati all'interno dei file senza introdurre meccanismi avanzati di gestione transazionale o controllo dello stato delle tabelle. Di conseguenza, nelle architetture tradizionali basate solo su file format, i motori di interrogazione dovevano scansionare intere directory del file system distribuito o dell'object storage per determinare quali file appartenessero a una determinata tabella, causando inefficienze prestazionali e seri problemi di consistenza in caso di modifiche concorrenti.

In questo contesto emergono gli Open Table Format, che risolvono alla radice tale limite spostando la definizione di "verità" della tabella dal percorso fisico della directory a una gestione centralizzata basata sui metadati.

Infatti, tali tecnologie introducono un layer logico di gestione sopra i file presenti nello storage distribuito. A differenza quindi dei semplici file format, un table format non si limita a descrivere come i dati vengono memorizzati, ma definisce anche:

- la struttura logica della tabella;
- la gestione dei metadati;
- le operazioni transazionali;
- il versionamento dei dati.

Gli Open Table Format rappresentano il fondamento tecnologico del paradigma Lakehouse, architettura nata per combinare la flessibilità e la scalabilità dei Data Lake con le funzionalità tipiche dei Data Warehouse tradizionali. [AGX⁺21]

2.4.2 Il ruolo del Metadata Layer

Il nucleo tecnologico di un Open Table Format è il *metadata layer*. Questo strato è costituito da un insieme strutturato di file che tracciano e documentano lo stato e la composizione dei file fisici presenti nello storage (in formato Parquet, ORC o Avro).

Quando un motore di calcolo interagisce con il sistema, non dialoga direttamente con i file di dati grezzi, ma interroga il metadata layer. Quest'ultimo astrae la complessità dei file fisici e fornisce una vista tabellare coerente, ottimizzando l'accesso e garantendo l'integrità del dato.

2.4.3 Funzionalità Chiave dell'Architettura

Gli Open Table Format introducono una struttura *metadata-driven* in cui ogni modifica alla tabella genera una nuova versione consistente dello stato dei dati. Questo modello consente di implementare proprietà ACID anche in ambienti distribuiti basati su object storage, garantendo atomicità e isolamento delle operazioni.

Tra le principali funzionalità abilitate dagli Open Table Format vi sono:

- operazioni CRUD transazionali
- supporto a operazioni MERGE incrementali
- snapshot isolation
- partition evolution
- time travel e query storiche

In particolare, il time travel permette di accedere a versioni precedenti della tabella attraverso snapshot consistenti.

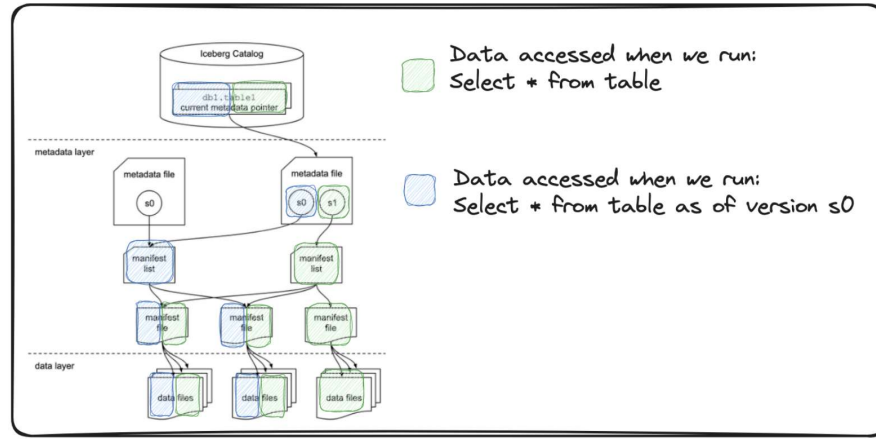


Figure 2.7: Time Travel in Open Table Formats. [Mac23]

La schema evolution consente invece di modificare la struttura logica delle tabelle senza richiedere la riscrittura completa dei dataset, migliorando la flessibilità delle pipeline ETL.

Le principali implementazioni Open Table Format diffuse nel panorama Lakehouse sono Apache Iceberg, Delta Lake e Apache Hudi. Sebbene condividano l'obiettivo di introdurre funzionalità transazionali nei Data Lake, tali tecnologie adottano approcci differenti nella gestione dei metadati, delle operazioni incrementali e della compatibilità con i diversi motori di elaborazione.

Nel contesto di questa tesi, l'attenzione è rivolta principalmente ad Apache Iceberg.

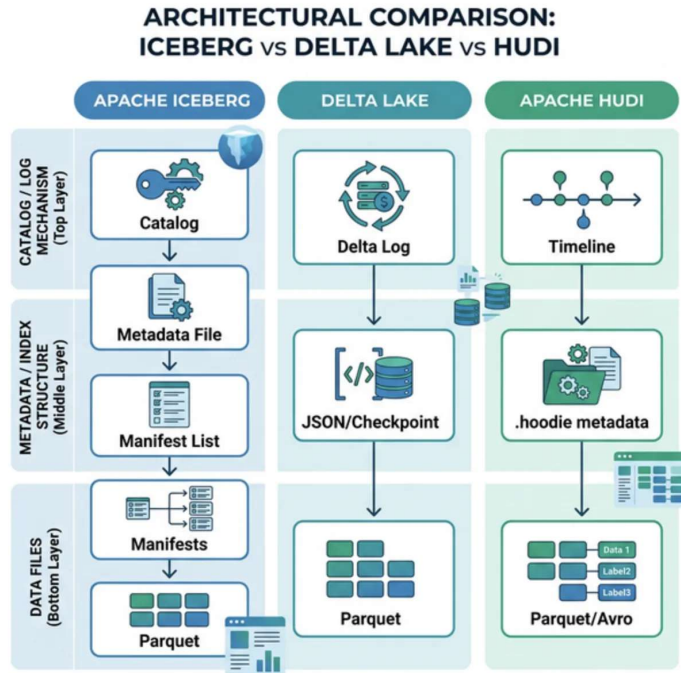


Figure 2.8: Open Table Formats Architectures. [Mer23]

2.5 Apache Iceberg: Architettura e funzionalità

Iceberg rappresenta infatti una delle implementazioni più mature del paradigma Lakehouse, introducendo una struttura gerarchica di metadati e snapshot che consente di separare il layout fisico dei file dalla rappresentazione logica della tabella [Apa25b].

2.5.1 Architettura dei metadata

Descrizione dettagliata del layer di metadata di Iceberg, con focus su Catalog, Metadata File, Manifest List e Manifest File, Snapshot.

2.5.2 Operazioni transazionali e Snapshot

ACID, optimistic concurrency control, MERGE, snapshot isolation e idempotenza.

2.5.3 Time Travel e Schema Evolution

versionamento, query storiche, evoluzione schema, partition evolution.

2.5.4 Maintenance e ottimizzazione

Compaction, snapshot expiration, orphan cleanup, metadata growth.

2.5.5 Confronto con Delta Lake e Hudi

Tabella comparativa tra le tre soluzioni, con focus su metadata management, modalità di scrittura (MOR vs COW), interoperabilità, supporto per partition evolution e compatibilità con i principali motori di elaborazione.

2.6 Ecosistema AWS

Amazon S3 -i Data Lake Storage

le caratteristiche di S3 come object storage.

AWS Glue Catalog

Il ruolo del catalogo centralizzato per la memorizzazione dei metadati delle tabelle Iceberg

AWS Glue Jobs

L'utilizzo di Spark in modalità serverless per l'esecuzione delle pipeline ETL.

Amazon Athena

Gli strumenti per l'interrogazione ad-hoc dei dati.

Amazon S3 Tables

Nuova soluzione managed di AWS per la gestione di tabelle Iceberg.

Chapter 3

Analisi As-is: Architettura Esistente

- Pipeline ETL esistente: Descrizione del flusso dati basato su Spark e file Parquet.
- Pain Points: Analisi dettagliata delle inefficienze riscontrate, i 6 punti critici del documento di formazione.

Chapter 4

Progettazione e Implementazione

- Design della soluzione (To-Be): Come cambia l'architettura logica.
- Configurazione dello Stack: Integrazione di Iceberg con Spark su AWS.
- Ottimizzazione: Implementazione di processi di Maintenance (Compaction dei file, rimozione dei vecchi snapshot/orphan files).

Chapter 5

Test e risultati

- Benchmark di Performance: Confronto tra il vecchio sistema e il nuovo (tempi di esecuzione del batch).
- Affidabilità: Dimostrazione del supporto alle transazioni ACID (es. cosa succede se un job fallisce a metà).
- Costi: Impatto dell'ottimizzazione dei file (Compaction) sui costi di storage.
- Risposta ai 6 punti critici

Chapter 6

Conclusioni e sviluppi futuri

- Risultati raggiunti: Sintesi dei benefici ottenuti dall'azienda.
- Limiti riscontrati: Eventuali criticità del framework o dei servizi AWS durante lo sviluppo.
- Evoluzioni: aggiornamento parziale sui record compositi

Bibliography

- [Ach24] Alessandro Achilli. Data lakehouse: cos'è, architettura e confronto con data warehouse. <https://www.it-impresa.it/blog/data-lakehouse/>, 2024.
- [AGX⁺21] Michael Armbrust, Ali Ghodsi, Reynold Xin, Matei Zaharia, et al. Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of CIDR*, 8(1):28, 2021.
- [Ama25a] Amazon Web Services. Amazon s3 tables documentation. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables.html>, 2025.
- [Ama25b] Amazon Web Services. Aws glue documentation, 2025. <https://docs.aws.amazon.com/glue/>.
- [Apa25a] Apache Software Foundation. Apache iceberg documentation, 2025. <https://iceberg.apache.org>.
- [Apa25b] Apache Software Foundation. Apache iceberg specification. <https://apache.github.io/iceberg/spec/>, 2025.
- [Apa25c] Apache Software Foundation. Apache parquet documentation. <https://parquet.apache.org/docs/>, 2025.
- [Apa25d] Apache Software Foundation. Apache spark documentation. <https://spark.apache.org/docs/latest/>, 2025.
- [BGK21] Edmon Begoli, Ian Goethert, and Kathryn Knight. A lakehouse architecture for the management and analysis of heterogeneous data for

- biomedical research and mega-biobanks. In *2021 IEEE International Conference on Big Data (Big Data)*, pages 4643–4651. IEEE, 2021.
- [BN21] Vladimir Belov and Evgeny Nikulchev. Analysis of big data storage tools for data lakes based on apache hadoop platform. *International Journal of Advanced Computer Science and Applications*, 12(8), 2021.
- [CD97] Surajit Chaudhuri and Umeshwar Dayal. An overview of data warehousing and olap technology. *ACM Sigmod record*, 26(1):65–74, 1997.
- [DB26] Confluent Developer and Tim Berglund. Iceberg + tableflow: Confluent course on apache iceberg, 2026. YouTube playlist, <https://www.youtube.com/playlist?list=PLf38f5LhQthcLN84xP6JgQ-pEbAc8SbSi>.
- [el] exact lab. Data lake in cloud. <https://www.exact-lab.it/en/data-lake-and-iot-simple-safe-efficient/>.
- [Inm95] William H Inmon. What is a data warehouse? *Prism Tech Topic*, 1(1):1–5, 1995.
- [Mac23] Joseph Machado. What is an open table format? why to use one? https://www.startdataengineering.com/post/what_why_table_format/, 2023.
- [Mer23] Alex Merced. Exploring the architecture of apache iceberg, delta lake, and apache hudi. <https://www.dremio.com/blog/exploring-the-architecture-of-apache-iceberg-delta-lake-and-apache-hudi/>, 2023.
- [Nix24] Michael Nixon. Data warehouse, data lake e data lakehouse: Tutto quello che c'è da sapere. <https://www.snaplogic.com/it/blog/data-warehouses-data-lakes-data-lakehouses-everything-you-need-to-know/>, 2024.
- [NS25] Srinivasa Rao Nelluri and FA Albert Saldanha. Mastering big data formats: Orc, parquet, avro, iceberg, and the strategy of selection.

BIBLIOGRAPHY

- International Journal of Computer Trends and Technology*, 73(1):44–50, 2025.
- [Sah24] Dipankar Saha. Disruptor in data engineering-comprehensive review of apache iceberg. *SSRN Electronic Journal*, 2024.
- [SGL⁺24] Jan Schneider, Christoph Gröger, Arnold Lutsch, Holger Schwarz, and Bernhard Mitschang. The lakehouse: State of the art on concepts and technologies. *SN Computer Science*, 5(5):449, 2024.
- [Sta24] Databricks Staff. What is a data lakehouse? <https://www.databricks.com/blog/what-is-data-lakehouse>, 2024.
- [Sta25] Qlik Staff. What is parquet file format? use cases and benefits. <https://www.qlik.com/blog/what-is-the-parquet-file-format-use-cases-and-benefits>, 2025.

Acknowledgements

Optional. Max 1 page.